

## L'audio del DMD

---

Una scheda USB, un avviso per servizio, gli effetti dei giochi, Doom e il Game Boy

### 1. Perché serve una scheda USB

---

Non è un ripiego, ed è la prima cosa da chiarire perché sembra una scelta discutibile e invece non è una scelta.

La libreria che pilota la matrice si prende il **blocco PWM** del Raspberry. L'audio analogico interno — il jack, e l'uscita `bcm2835` che compare come scheda 0 — usa lo stesso blocco. Sono alternativi per costruzione: infatti l'installazione del DMD spegne il modulo `snd_bcm2835`, e senza spegnerlo la libreria della matrice si rifiuta di partire.

Quindi: o suona il jack, o si accende il pannello. Non entrambi.

Una scheda audio USB è un dispositivo completamente diverso. Parla con `snd_usb_audio`, non tocca il PWM, e convive col pannello senza nemmeno saperlo. È **l'unico** modo di avere suono su questa macchina.

Vanno bene le chiavette da pochi euro con chip C-Media o simili: quello che serve è un'uscita stereo. Si infila e basta — il kernel la riconosce da sola, e compare come scheda 1 (o 2, se ce n'è già una).

Per verificarlo: `cat /proc/asound/cards`. Se la chiavetta c'è, è in quell'elenco. È lo stesso file che legge il DMD.

### 2. Chi suona

---

`ffmpeg`, che sul DMD c'è già dalla versione 1.1 — lo usa il Media Player per i video.

Fra le sue uscite ha il muxer `alsa`, quindi:

```
ffmpeg -i suono.mp3 -filter:a volume=0.6 -f alsa plughw:1,0
```

riproduce wav e mp3 su una scheda scelta, col volume voluto, **senza installare niente di nuovo**. `aplay` non legge gli mp3 e non regola il volume; `ffplay` si tira dietro SDL. Il ferro del mestiere lo avevamo già in casa.

`plughw`, **non** `hw`

Dettaglio piccolo e decisivo. `hw:1,0` è la scheda nuda: accetta solo la frequenza e il formato che sa fare. `plughw:1,0` è la stessa scheda con un convertitore davanti.

Un wav a 44 100 Hz stereo su una chiavetta che vuole 48 000 Hz mono, con `hw`, non parte e basta — e il motivo non si scopre in fretta. Con `plughw` esce, e la conversione la fa ALSA. Il DMD usa sempre `plughw`.

### 3. Le quattro cose, che sono davvero quattro

---

Le impostazioni dell'audio sono divise perché rispondono a domande diverse, non per gusto della simmetria. I due emulatori si portano il proprio suono; il resto lo mettiamo noi.

|                           | Cosa suona                             | Da dove viene il file                                                       |
|---------------------------|----------------------------------------|-----------------------------------------------------------------------------|
| <b>Avvisi dei servizi</b> | quando una sorgente prende il pannello | libreria media, scelto da te                                                |
| <b>Effetti dei giochi</b> | Invaders e Breakout                    | <code>suoni/</code> , dentro il programma                                   |
| <b>Musica dei giochi</b>  | il sottofondo di una partita           | <code>suoni/</code> , <code>/var/lib/dmd/suoni</code> , o la libreria media |
| <b>Audio di Doom</b>      | la colonna sonora del gioco            | i WAD, come nel gioco vero                                                  |
| <b>Audio del Game Boy</b> | la musica della cartuccia              | l'APU emulata da PyBoy                                                      |

### 3.1 Gli avvisi dei servizi

Il momento è **quello della notifica**: l'istante in cui una sorgente vince l'arbitrato e prende il pannello. Non mentre ci resta, non a intervalli. Una volta, quando compare.

Per ogni servizio, nella pagina **Servizi**, c'è un menu con i file `wav` e `mp3` presenti nella libreria media, e un pulsante **Ascolta** per sentirlo prima di sceglierlo. «Nessun suono» toglie l'assegnazione.

I servizi che possono avere un avviso sono undici:

`zedmd` · `nowplaying` · `birthdays` · `air_radar` · `clock` · `scadenze` · `calendario` · `webcam` · `notifiche` · `info` · `notifiche` · `avviso` · `notifiche` · `allarme`

**Le notifiche da Home Assistant ne hanno tre, uno per livello.** È l'unico servizio con più di un campanello, e il motivo è che una notifica arriva mentre non stai guardando il pannello: un suono solo direbbe «è successo qualcosa» e ti obbligherebbe ad andare a vedere, tre dicono dall'altra stanza se vale la pena alzarsi. Nella pagina portano in coda *MQTT / Home Assistant*, perché in un elenco di servizi del pannello sono le uniche tre righe che arrivano da fuori.

**Tre non ce l'hanno, e non è una dimenticanza.** Media Player e Rolling Banner non annunciano niente: compaiono a intervalli per decorazione, e un campanello a ogni foto sarebbe un metronomo in soggiorno. I rifiuti non compaiono per un motivo diverso: non sono un servizio. Li disegna l'orologio, dentro la sua colonna, e non prendono mai il pannello per conto proprio.

Un avviso viene **troncato a 15 secondi**. Un file lungo scelto per sbaglio terrebbe muto tutto il resto per minuti.

#### Un suono per volta

Se un avviso parte mentre un altro sta ancora suonando, il nuovo si **scarta**. Non va in coda e non interrompe quello in corso.

È una decisione, non una limitazione tecnica. Un avviso in coda si farebbe sentire tre secondi dopo, riferito a una cosa che dal pannello è già sparita: peggio che non sentirlo. E interromperne uno a metà è un singhiozzo.

### 3.2 Gli effetti dei giochi

Invaders e Breakout hanno diciassette effetti a onda quadra, 22 050 Hz mono, scritti per loro. Stanno in `/opt/dmd/suoni/` — **dentro il programma, non nella libreria media**.

La differenza conta: sono parte del gioco come lo sono i suoi colori. Mescolarli ai tuoi contenuti vorrebbe dire che cancellandone uno per sbaglio il gioco diventa muto, e che comparirebbero nel menu degli avvisi come se fossero roba da scegliere.

| Effetto                                         | Quando                                                   |
|-------------------------------------------------|----------------------------------------------------------|
| <code>sparo</code>                              | Invaders: il colpo del cannone                           |
| <code>colpito</code>                            | Invaders: un alieno esplode                              |
| <code>passo1</code> ... <code>passo4</code>     | Invaders: le quattro note del passo della schiera        |
| <code>mattone1</code> ... <code>mattone5</code> | Breakout: un mattone si rompe — <b>una nota per fila</b> |
| <code>racchetta</code>                          | Breakout: rimbalzo sulla racchetta                       |
| <code>muro</code>                               | Breakout: rimbalzo sulle pareti                          |
| <code>lancio</code>                             | Breakout: la palla parte dalla racchetta                 |
| <code>livello</code>                            | entrambi: schiera ripulita o muro finito                 |
| <code>persa</code>                              | entrambi: vita persa                                     |
| <code>record</code>                             | entrambi: primato personale battuto                      |

Due sono citazioni, non decorazioni.

Le **quattro note del passo** di Invaders sono la cosa più fedele all'originale del 1978: è il battito del gioco, e accelera insieme alla schiera. Metà della tensione stava lì.

Le **cinque note dei mattoni** di Breakout salgono man mano che si scava verso l'alto — la fila in cima, quella che vale 50 punti, è la più acuta. Nel Breakout del 1976 era il modo in cui il gioco diceva, senza scriverlo, a che punto eri arrivato.

Breakout resta comunque più rado di Invaders, e non è un difetto: misurati 131 suoni al minuto contro 291. Invaders ha una cadenza continua, Breakout solo eventi.

### Il mixer (dalla 5.6)

Fino alla 5.5 ogni effetto era un processo a sé, e una scheda ALSA aperta in `plughw` sta in mano a un programma alla volta: due suoni ravvicinati non potevano convivere, e il secondo veniva **buttato**. Su Breakout se ne perdeva circa un quinto — abbastanza da sentire mattoni muti.

Durante una partita ora c'è un riproduttore solo, aperto quanto dura la partita, e gli effetti si **sommano in memoria**: si sovrappongono invece di annullarsi, e partono nel blocco successivo (23 ms) invece di pagare i 150-300 ms che costa avviare un processo su un Pi.

Vive solo mentre si gioca. A pannello fermo il mixer non esiste, non tiene occupata la scheda e non consuma niente — e gli avvisi dei servizi continuano a funzionare come sempre, con la loro regola del suono per volta.

Si spengono tutti insieme dalla levetta **Effetti dei giochi** in Impostazioni.

## 3.3 Doom

Doom suona con la sua musica e i suoi suoni, presi dal WAD, esattamente come nel gioco originale.

Fino alla 5.1 il DMD lo lanciava sempre con `-nosound -nomusic`. Ora quei due argomenti compaiono solo se l'audio di Doom è spento.

**Serve una ricompilazione, e fino alla 5.4 questa pagina mentiva.** La 5.2 ha aggiunto la levetta e questo manuale, ma il binario non aveva **nessun modulo sonoro dentro**: il Makefile non ne compilava, e in cima c'era scritto a chiare lettere «non vuole SDL e non fa suono». Togliere `-nosound` non poteva bastare, e infatti non bastava.

Dalla 5.5 il Makefile compila il modulo sonoro di doomgeneric (`i_sdl_sound.c`, `i_sdl_music.c`) quando trova **SDL2** e **SDL2\_mixer**. Se non li trova compila muto come prima, invece di fallire.

Chi aggiorna deve **ricompilare**: pagina Doom → *Prepara Doom*. Sono un paio di minuti e installa da sé le due librerie. L'aggiornamento via rete non ricompila apposta — lo farebbe a ogni update, per niente. La pagina Doom controlla che cosa ha collegato il binario e lo dice da sola.

*Nota della 5.5.1.* La prima ricompilazione poteva fallire con `multiple definition of 'I_InitTimidityConfig'`. Non era la libreria: `-DFEATURE_SOUND` cambia il significato dei sorgenti (`dummy.c` definisce quello stub proprio quando la flag manca) ma non tocca nessun `.c`, quindi make ricompilava solo i file nuovi e collegava oggetti incoerenti. Ora gli oggetti dipendono anche dalle opzioni e si ricompila tutto una volta sola.

La scheda gliela passiamo dall'ambiente, non dalla riga di comando:

```
SDL_AUDIODRIVER=alsa
AUDIODEV=plughw:1,0
```

SDL sceglie il dispositivo da `AUDIODEV`; senza `SDL_AUDIODRIVER=alsa` proverebbe prima pulseaudio, che su un'immagine senza sessione grafica non c'è, e Doom partirebbe muto senza spiegare perché.

### Perché ha una levetta tutta sua

Perché è l'unico suono **continuo** del sistema, e quindi l'unico che costa qualcosa.

Riprodurre audio è CPU e traffico USB, cioè la stessa moneta con cui si pagano le righe chiare sul pannello — il disturbo che la pagina Taratura misura. Un avviso di mezzo secondo non si misura nemmeno. Una colonna sonora che va per tutta la partita sì. Se durante Doom noti disturbi che senza non ci sono, la levetta è lì per questo.

## 3.4 Il Game Boy

Il Game Boy suona con il **suo** chip sonoro, emulato da PyBoy. Non ci sono effetti nostri: la musica di un gioco Game Boy è scritta per quell'APU, e rifarla vorrebbe dire rifare il gioco.

Il meccanismo è diverso da quello di Doom, e più semplice. PyBoy con `sound_emulated=True` **calcola** i campioni senza aprire nessun dispositivo audio, e dopo ogni tick te li lascia leggere: int8 stereo a 48 kHz, circa 800 campioni per fotogramma. Il DMD li converte a 16 bit e li scrive nella pipe di un ffmpeg che fa da uscita ALSA — lo stesso attrezzo di tutto il resto, e nessuna dipendenza nuova per PyBoy.

Un dettaglio che conta: i campioni si raccolgono **a ogni tick**, anche dai fotogrammi che non vengono disegnati. Il DMD ne mostra 30 al secondo dei 59,7 che il Game Boy produce, perché l'occhio su 64 righe non vede la differenza e la pipe video costa. L'orecchio invece la sentirebbe eccome: prendere i campioni solo dai fotogrammi mostrati vorrebbe dire buttare via due terzi del suono.

Non ha una levetta propria: segue quella degli **effetti dei giochi**. E tace quando la scheda è occupata dalla musica AirPlay, come tutto il resto.

### La latenza, e perché serve `aplay`

Nella 5.5 l'audio del Game Boy arrivava **in ritardo** rispetto all'immagine, e la colpa era di ffmpeg — ma non di un suo difetto.

Il muxer `alsa` di ffmpeg **non accetta nessuna opzione**: apre un buffer fisso di 32768 campioni, che a 48 kHz fanno **0,68 secondi**. Aggiungi i 340 ms del tubo Linux, che di suo

tiene 64 KB, e il conto è fatto. Per un campanello di mezzo secondo è irrilevante; per un emulatore è la distanza fra quello che si vede e quello che si sente.

Dalla 5.6 il PCM va a `aplay`, che il buffer lo prende come argomento — **80 ms** — e il tubo si stringe a 16 KB. `aplay` sta in `alsa-utils`, lo stesso pacchetto di `alsamixer` che questo manuale ti fa già usare per alzare il volume della chiavetta: su Raspberry Pi OS c'è quasi sempre. Se manca, si ripiega su `ffmpeg` — in ritardo, ma non muto.

Vale per il Game Boy e per il mixer degli effetti. Gli avvisi dei servizi restano su `ffmpeg`: leggono mp3, che `aplay` non sa fare, e mezzo secondo di buffer su un campanello non lo nota nessuno.

## 4. Il DMD come cassa AirPlay

Questo sta nella pagina **Now Playing**, non in Impostazioni, perché è una proprietà della musica e non del suono in generale.

### Che cosa cambia

`setup_nowplaying.sh` configura `shairport-sync` per scrivere l'audio nella scheda **fittizia** del kernel (`snd_dummy`). Non era un capriccio: senza una scheda audio l'audio andava buttato da qualche parte, e serviva un dispositivo con un orologio vero — `/dev/null` e il plugin `null` di ALSA non limitano il ritmo, e `shairport-sync` perderebbe il riferimento temporale.

Dei brani il DMD prendeva quindi solo i metadati: titolo, artista, avanzamento. Il suono lo buttava.

Con una scheda USB collegata quella scelta non ha più motivo di esistere. La spunta «**La musica esce dalla scheda audio**» sposta l'uscita di `shairport-sync` sulla scheda vera, e il pannello diventa una cassa AirPlay a tutti gli effetti: qualunque cosa parta da un iPhone, un iPad o un Mac — Apple Music, Spotify, YouTube, un video — suona qui.

Meccanicamente è **una riga** di `/etc/shairport-sync.conf` più un riavvio del servizio.

### Solo AirPlay

Le tre sorgenti di Now Playing non sono la stessa cosa, e qui la differenza diventa concreta:

| Sorgente        | Che cos'è                                                | Si può far uscire? |
|-----------------|----------------------------------------------------------|--------------------|
| <b>airplay</b>  | un flusso audio che arriva davvero qui                   | <b>sì</b>          |
| <b>spotify</b>  | l'API web dice cosa suona <i>su un altro dispositivo</i> | no                 |
| <b>external</b> | metadati di un'automazione                               | no                 |

Spotify sta raccontando una cosa che sta succedendo altrove: non c'è nessun audio da dirottare. Per fare del DMD anche un altoparlante Spotify Connect servirebbe `librespot`, che è un altro programma, un'altra installazione e un'altra storia.

### La prova prima di scrivere

Accendendo l'interruttore senti un tono breve. Non è un vezzo: è la verifica.

`shairport-sync` riceve il dispositivo in forma **esclusiva** (`hw:1,0`), non attraverso il convertitore (`plughw:1,0`) che usiamo per gli avvisi. Sembra un'incoerenza e non lo è: `plughw` è quello che vogliamo per un mp3 qualunque, mentre `shairport-sync` la frequenza la gestisce da sé e la scheda vuole vederla com'è — con `plughw` gli si nasconde ciò che sa fare davvero, e la sincronizzazione ne risente.

Ma `hw:` può rifiutare. Quindi prima di scrivere la configurazione il DMD suona un tono nel formato esatto di AirPlay — **44 100 Hz, stereo, 16 bit** — su quel dispositivo. Se la scheda non lo regge, **non si tocca niente** e il motivo compare in pagina. Meglio un interruttore che non scatta di una cassa AirPlay che non suona più e nessuno sa perché.

Della configurazione precedente resta sempre una copia `.bak` accanto, e viene cambiata

**una sola riga:** nello stesso file c'è la password del broker MQTT, e riscriverlo da capo vorrebbe dire perderla o maneggiarla.

## Chi tiene la scheda mentre suona la musica

Una scheda ALSA aperta in `hw:` sta in mano a un programma alla volta. Mentre c'è un brano in corso, quel programma è `shairport-sync`.

Quindi **durante la musica gli avvisi dei servizi tacciono**, e non ci provano nemmeno: il DMD lo sa, perché è la stessa informazione che sta disegnando sul pannello. Le notifiche si vedono lo stesso — compleanni, scadenze, aerei — solo non si sentono.

È la scelta giusta anche a prescindere dal vincolo tecnico: un campanello sopra la musica non lo vuole nessuno, e se stai ascoltando qualcosa, quella cosa ha la precedenza.

**Doom**, per la stessa ragione, parte muto se lo si lancia mentre la musica va. Deciso da noi, invece di lasciare che SDL fallisca l'apertura della scheda con qualche secondo di errori proprio all'inizio della partita. Se vuoi l'audio del gioco, ferma la musica e ricomincia la partita.

## Spegnendolo

Si torna alla scheda fittizia. Now Playing continua a mostrare i brani esattamente come prima: semplicemente non li suona.

## 5. La pagina Impostazioni

Il riquadro **Audio** ha cinque comandi.

**Altoparlante attivo** — l'interruttore generale. Spento, non suona niente: né avvisi, né giochi, né Doom.

**Uscita audio** — le schede viste dal kernel, lette da `/proc/asound/cards`. Lasciando «L'ultima collegata» il DMD sceglie da sé, e la regola ha **due livelli**: prima le schede vere e proprie — una chiavetta USB, un DAC — e solo se non ce n'è nessuna le uscite interne del Raspberry. Dentro ogni livello vale l'ultima della lista, che con una chiavetta appena infilata è quasi sempre quella giusta.

**Perché due livelli e non uno.** Fino alla 7.3 la regola era «l'ultima scheda non fittizia», scritta quando in quell'elenco c'erano solo la scheda finta e la chiavetta. Su un Pi 4 con Now Playing installato l'elenco vero è:

```
0 [Dummy    ]: Dummy - Dummy
1 [Audio     ]: USB-Audio - USB Audio
2 [vc4hdmi0 ]: vc4-hdmi - vc4-hdmi-0
3 [vc4hdmi1 ]: vc4-hdmi - vc4-hdmi-1
```

Le due uscite HDMI si registrano **dopo** la USB, quindi «l'ultima» era `plughw:3,0`: ogni avviso finiva in un'uscita HDMI a cui non è collegato niente. Silenzio perfetto, e nessun errore da leggere.

Le uscite interne restano in elenco e si possono scegliere a mano — su un'altra macchina potrebbero servire davvero — ma non sono mai la scelta automatica. L'HDMI vuole un cavo che qui non c'è, e il jack non può funzionare comunque: la libreria della matrice si prende lo stesso blocco PWM, e l'installazione lo disattiva apposta (§ 1).

Se la scheda che avevi scelto viene staccata, il DMD **non ripiega su un'altra**: resta muto e lo dice in pagina. Suonare dall'altoparlante sbagliato senza avvisare è peggio che non suonare, perché non si capisce cosa stia succedendo.

**Volume effetti sonori servizi** — quanto forte parla il DMD di sua iniziativa: un aereo, un compleanno, una notifica. Non tocca il mixer di sistema: è un filtro `volume=` applicato da `ffmpeg` al momento della riproduzione, quindi non cambia niente per gli altri programmi e non ha bisogno di `alsamixer`.

**Il cursore segue l'orecchio, non l'ampiezza.** Fino alla 12.3 il numero salvato era il guadagno, e il cursore sembrava rotto: «*ho abbassato il volume ma sembra non avere effetto*». Il numero arrivava a ffmpeg giusto — misurato, a metà cursore uscivano  $-6$  dB esatti — ma metà cursore non è metà volume: l'orecchio chiama «metà» qualcosa come  $-10$  dB, e la metà alta della corsa non serviva a niente, perché da 100 a 70 sono 3 dB che nessuno sente.

Dalla 12.4 in configurazione sta **la posizione del cursore** e il guadagno lo dà una curva quadratica:

| cursore | guadagno | dB    |
|---------|----------|-------|
| 100     | 1,00     | 0     |
| 70      | 0,49     | $-6$  |
| 50      | 0,25     | $-12$ |
| 30      | 0,09     | $-21$ |
| 10      | 0,01     | $-40$ |

Gli estremi non si muovono: 0 è silenzio, 100 è fondo scala. La curva vale per tutte e tre le strade del suono — ffmpeg per gli avvisi, il mixer per i giochi integrati, la pipe dei tasti per Doom e il Game Boy — perché un cursore che si comporta in due modi è peggio di due cursori. Nella pagina e in Home Assistant si continua a leggere la posizione, non il guadagno: altrimenti il numero scenderebbe da solo a ogni salvataggio.

Effetto pratico dell'aggiornamento: a parità di cursore tutto suona un po' più piano di prima. È il prezzo di avere una corsa che serve tutta.

**Volume dei giochi** — separato dal precedente, e non per capriccio. Il volume degli avvisi lo si abbassa pensando al pannello che parla da solo, magari di sera: su un DMD vero era a 0,05. Un effetto di un gioco invece risponde a un tasto appena premuto e dura cinquanta millesimi — e l'orecchio integra il volume su due decimi di secondo, quindi un suono così corto si sente molto più piano di uno lungo con la stessa ampiezza. Con una manopola sola, o gli avvisi urlano o i giochi spariscono. Il predefinito è 0,90, e chi aggiorna non eredita il volume degli avvisi.

Dalla 10.1 vale **per tutti i giochi**, e anche **a partita aperta**: lo si abbassa mentre si gioca e si abbassa subito.

- **Giochi integrati** — il volume entra negli effetti già preparati; cambiandolo si rifanno.
- **Doom** — lo riceve all'avvio e poi sulla stessa pipe dei tasti. Il 100% è il volume di serie di Doom, che misurato sta alla pari dei nostri effetti a fondo scala. Fino alla 10.0 Doom non lo riceveva **affatto**: scriverlo nel suo `default.cfg` non sarebbe servito, perché doomgeneric quel file non lo legge — la lettura è dentro `#if ORIGCODE`, compilata fuori. Lo si è misurato registrando l'uscita audio: con `sfx_volume 0` nel file, effetti identici. Serve **ricompilare Doom** una volta (pagina Doom, «Prepara»): il binario vecchio non conosce il comando, e finché non lo si ricompila il volume non gli viene mandato — perché lo leggerebbe come un tasto premuto.
- **Doom parte già al volume giusto** dalla 10.5.1: prima, per il secondo e mezzo dell'avvio, suonava al volume di serie.
- **Game Boy** — fino alla 10.0 usava per sbaglio il volume **degli avvisi**: di notte quello è il volume notturno, cioè zero, e il Game Boy giocava muto.

**La musica dei giochi**, dalla 10.9, **riscritta nella 11.3**. Ogni gioco integrato ha un suo motivo di sottofondo, scritto apposta per questo progetto. Dalla 11.3 sono musiche **da sala giochi**: sotto ognuna ci sono una batteria (cassa sull'uno e sul tre, rullante sul due e sul quattro, charleston sui controtempi), un basso a **ottavi** — fondamentale e quinta alternate — e un **arpeggio a sedicesimi**, la seconda voce che dà il movimento. Dai 150 ai 172 battiti al minuto.

| Gioco        | Il motivo                                                                 |
|--------------|---------------------------------------------------------------------------|
| Breakout     | il più veloce, la minore: è un gioco di riflessi                          |
| Invaders     | pulsa, ma <b>senza rullante</b> : il ritmo è la marcia degli alieni       |
| Snake        | chip saltellante, tutto arpeggi                                           |
| Pongo        | il più discreto di tutti: il basso pulsa, ma il <i>ping</i> resta la voce |
| Squadriglia  | fanfara da cabinato, con cassa e rullante                                 |
| Gnam Gnam    | il solito motivo, suonato come in sala giochi                             |
| Mine vaganti | mi minore, arpeggio fitto: il campo minato si muove                       |
| T-Rex        | ottavi ribattuti, il passo della corsa                                    |
| Kingo Bongo  | tamburi in evidenza, melodia a botta e risposta                           |

Stanno tutti **sotto gli effetti** — da un quarto a un terzo del loro volume — tacciano a partita finita e, dove c'è una schermata iniziale, finché non si preme fuoco. Seguono il cursore **Volume dei giochi** e la levetta **Effetti dei giochi**; in più c'è la casella **Musica di sottofondo nei giochi** nella pagina Giochi, per chi vuole gli effetti ma non la musica. Le ricette stanno in `diagnostica/genera_suoni.py`.

**In Home Assistant**, dalla 10.6, i tre volumi — avvisi, giochi e notturno — sono tre cursori fra le entità del DMD (`number.dmd_volume_avvisi`, `number.dmd_volume_giochi`, `number.dmd_volume_notte`). Servono alle automazioni: abbassare gli avvisi la sera, alzare i giochi quando si gioca.

**Prova il suono** — suona un effetto sulla scheda scelta. Funziona **anche a suono spento**, apposta: serve proprio a capire se l'audio funziona *prima* di accenderlo. E usa sempre il volume di giorno: un pulsante di prova che di notte suona il silenzio non prova niente, anzi fa credere che la scheda sia rotta.

Accanto ci sono le due levette **Effetti dei giochi** e **Audio di Doom**, che sono i § 3.2 e § 3.3.

## Il fruscio che tiene sveglia la scheda

C'è una voce che non compare in nessuna pagina, `audio.sottofondo`, e vale la pena sapere perché esiste.

Molti convertitori USB — compreso quello con cui questo progetto è stato messo a punto — si **automutano** quando ricevono zero digitale esatto, e al primo campione diverso da zero riaprono l'uscita con una rampa di qualche decina di millesimi, per non fare il «plop». Un effetto di gioco dura cinquanta o settanta millesimi: arriva mentre la rampa è a metà e sparisce quasi tutto. Se invece un altro suono è appena passato, la scheda è sveglia e lo stesso effetto si sente benissimo — ed è esattamente il sintomo che si vedeva sul campo: *quando lo sento, lo sento bene*, con Snake che ne perdeva nove su dieci e Breakout che ne salvava la maggior parte.

La prova che lo ha inchiodato è stata la più semplice: lo stesso beep, dieci volte di fila, con in mezzo silenzio digitale oppure un fruscio inudibile. Col silenzio non si sentiva; col fruscio sì. E lo conferma dal lato opposto quello che sul DMD ha sempre funzionato — Doom, il Game Boy, la musica AirPlay: nessuno dei tre scrive mai zero, perché hanno tutti un flusso continuo.

Quindi durante una partita il mixer non scrive mai zero, ma rumore bianco a  $-60$  dBFS: 32 su 32767, sotto il rumore di fondo di qualunque stanza. Si spegne mettendo `audio.sottofondo` a zero.

## Se la scheda non sa fare 44100 Hz

AirPlay trasmette a **44100 Hz e a nient'altro**. Molte chiavette USB economiche non ce l'hanno: si vede in un comando, dove `1` è il numero della scheda.

```
cat /proc/asound/card1/stream0
```

Se sotto `Playback:` la riga `Rates:` non contiene 44100 — per esempio `Rates: 8000, 48000` — quella scheda in accesso esclusivo non può riprodurre musica AirPlay, mai, in nessuna configurazione.

Il DMD prova prima l'accesso esclusivo (`hw:`, § 2) e, se fallisce, riprova la stessa scheda con il **convertitore di ALSA** davanti (`plughw:`). Con il convertitore shairport-sync non vede più che cosa la scheda sappia fare, e ci rimette un filo di precisione nella sincronizzazione fra più casse — che con una cassa sola non esiste. L'alternativa era il silenzio, quindi si accende e lo si dice nella pagina.

Se anche con il convertitore non suona, l'interruttore **non scatta** e sotto compare la riga di errore di ffmpeg. Meglio un interruttore che non si accende di una cassa che non suona e nessuno sa perché.

## Il volume in modalità notte

Nel riquadro **Night mode e Sleep mode** c'è un quarto campo accanto alla luminosità notturna: **Volume %**, predefinito **0**, cioè muto.

Vale solo se il Night mode è acceso, e riguarda quello che il pannello dice **di sua iniziativa** — un aereo di passaggio, un compleanno, una notifica. Non riguarda una partita: quella è una cosa che stai facendo tu adesso, e zittirla sarebbe come spegnere il pannello a chi ci sta giocando davanti. È la stessa eccezione che lo Sleep mode fa già per chi tiene il pannello occupato.

**Sleep mode e display spento non compaiono in questo discorso perché sono già silenziosi**, e non per una regola sull'audio: lì il ciclo principale si ferma prima di scegliere una sorgente, e l'avviso nasce proprio nel momento in cui una sorgente prende il pannello. Se il pannello non lo prende nessuno, non c'è niente da annunciare. Il Night mode era l'unica delle tre fasce in cui alle tre di notte un aereo di passaggio suonava a volume pieno.

Se apri la pagina mentre il Night mode è in corso, lo slider del volume continua a mostrare **il valore di giorno**, con un avviso sopra che spiega cosa sta succedendo. Non è una distrazione: se mostrasse il volume in vigore, aprire la pagina di notte e premere Salva scriverebbe zero in configurazione per sempre — che è esattamente l'inganno dello slider della luminosità durante il Night mode, già pagato una volta in questo progetto.

Se ffmpeg fallisce — scheda staccata a metà, file illeggibile — l'ultima riga del suo errore compare sotto il riquadro. Il silenzio non resta mai senza spiegazione.

## 6. Cosa non fa

---

Vale la pena elencarlo, perché sono assenze volute.

- **Non mixa.** Un suono per volta, il secondo si scarta (§ 3.1).
- **Non regola il volume di sistema.** Il filtro di ffmpeg vale per il DMD e basta.
- **Non tiene una libreria di suoni propria** per gli avvisi: i file sono i tuoi, nella cartella media, gestiti come le foto e i video — da SMB, o dall'upload della pagina Media.
- **Non annuncia i rifiuti, il Media Player e il Rolling Banner** (§ 3.1).
- **Non parla.** Nessuna sintesi vocale: sarebbe un'altra dipendenza e un'altra storia.

## 7. Se non si sente niente

---

In ordine, dal più probabile.

1. **La scheda c'è?** `cat /proc/asound/cards`. Se la chiavetta non compare, il problema è prima del DMD.
2. **È selezionata?** Impostazioni → Audio → Uscita. Se la voce scelta non esiste più, il riquadro lo dice.
3. **Il volume è a zero?** Sia quello del DMD, sia quello della scheda: `alsamixer -c 1` e alza il canale principale, che su molte chiavette parte basso o muto. È l'unico posto

dove serve `alsa-utils`.

4. **La prova suona?** Se sì, il problema è nell'assegnazione dei file: guarda il menu del servizio nella pagina Servizi.
5. **Se ffmpeg si lamenta**, il motivo è sotto il riquadro Audio. `Device or resource busy` vuol dire che un altro programma tiene la scheda; su questa macchina l'unico candidato è Doom.

Da riga di comando, per escludere il DMD dal ragionamento:

```
ffmpeg -f lavfi -i "sine=frequency=440:duration=1" -f alsa plughw:1,0
```

Se questo non suona, non è il DMD.